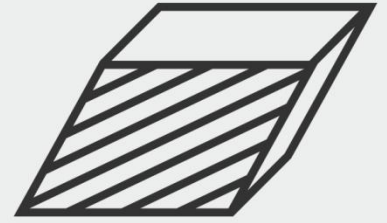




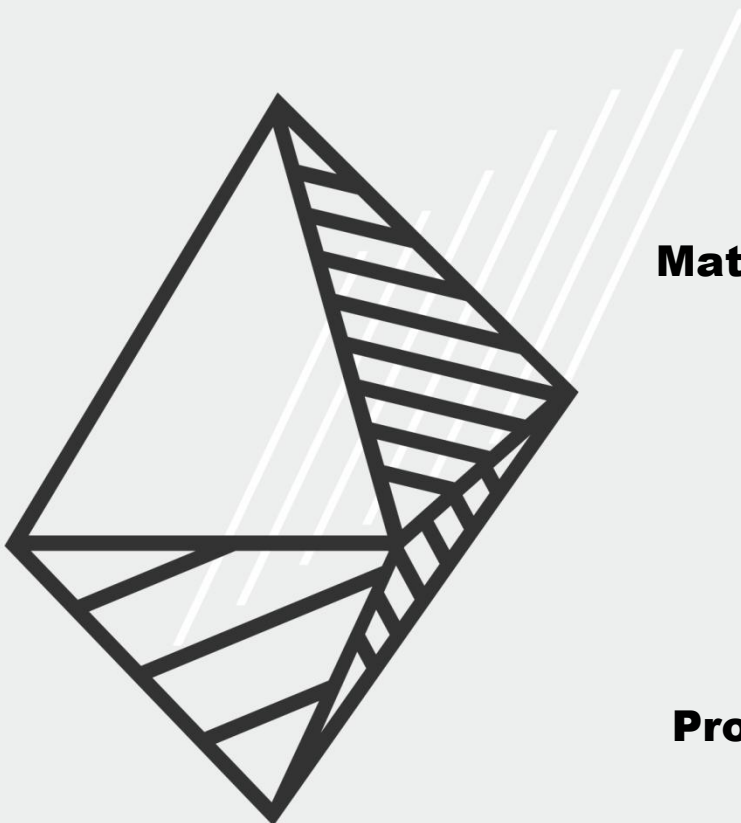
Fecha: 5 de Mayo del 2026



Grupo: 2a

Practica 11

Alumno: Issac Martin Recendiz



**Materia: Programacion Orientada
a Objetos**

Profesor: Jose Jesus Collazo



INTRODUCCIÓN

La herencia es un concepto de la Programación Orientada a Objetos que permite crear nuevas clases a partir de una clase base, reutilizando atributos y métodos.

En esta práctica se implementa un sistema de personajes para un videojuego utilizando herencia en C++. Se desarrolla una clase base que contiene características generales y varias clases derivadas que representan distintos tipos de personajes con habilidades específicas.

El objetivo es aplicar la reutilización de código y la sobrescritura de métodos para diferenciar el comportamiento de cada personaje.

```
#include <iostream>
using namespace std;

// Clase base
class Personaje {
protected:
    string nombre;
    int nivel;
    int vida;

public:
    Personaje(string n, int niv, int v) {
        nombre = n;
        nivel = niv;
        vida = v;
    }

    void mostrarInfo() {
        cout << "Nombre: " << nombre << endl;
        cout << "Nivel: " << nivel << endl;
        cout << "Vida: " << vida << endl;
    }

    virtual void atacar() {
        cout << nombre << " realiza un ataque básico." << endl;
    }
};

// Clase Guerrero
class Guerrero : public Personaje {
public:
    Guerrero(string n, int niv, int v) : Personaje(n, niv, v) {}

    void atacar() override {
        cout << nombre << " lanza un ataque fuerte con espada." << endl;
    }
};

// Clase Mago
```

```

class Mago : public Personaje {
public:
    Mago(string n, int niv, int v) : Personaje(n, niv, v) {}

    void atacar() override {
        cout << nombre << " lanza un hechizo mágico." << endl;
    }
};

// Clase Arquero
class Arquero : public Personaje {
public:
    Arquero(string n, int niv, int v) : Personaje(n, niv, v) {}

    void atacar() override {
        cout << nombre << " dispara una flecha a distancia." << endl;
    }
};

// MAIN
int main() {
    Guerrero g("Thor", 10, 100);
    Mago m("Merlin", 12, 80);
    Arquero a("Legolas", 11, 90);

    cout << "=== INFORMACIÓN ===" << endl;
    g.mostrarInfo();
    cout << endl;
    m.mostrarInfo();
    cout << endl;
    a.mostrarInfo();

    cout << "\n=== ATAQUES ===" << endl;
    g.atacar();
    m.atacar();
    a.atacar();

    return 0;
}

```

```
// Clase base
class Personaje {
protected:
    string nombre;
    int nivel;
    int vida;

public:
    Personaje(string n, int niv, int v) {
        nombre = n;
        nivel = niv;
        vida = v;
    }

    void mostrarInfo() {
        cout << "Nombre: " << nombre << endl;
        cout << "Nivel: " << nivel << endl;
        cout << "Vida: " << vida << endl;
    }

    virtual void atacar() {
        cout << nombre << " realiza un ataque básico." << endl;
    }
};
```

Captura 1: Clase Personaje

En esta parte se define la clase base Personaje, la cual contiene los atributos principales como nombre, nivel y vida, así como los métodos mostrarInfo() y atacar().

```
// Clase Guerrero
class Guerrero : public Personaje {
public:
    Guerrero(string n, int niv, int v) : Personaje(n, niv, v) {}

    void atacar() override {
        cout << nombre << " lanza un ataque fuerte con espada." << endl;
    }
};

// Clase Mago
class Mago : public Personaje {
public:
    Mago(string n, int niv, int v) : Personaje(n, niv, v) {}

    void atacar() override {
        cout << nombre << " lanza un hechizo mágico." << endl;
    }
};

// Clase Arquero
class Arquero : public Personaje {
public:
    Arquero(string n, int niv, int v) : Personaje(n, niv, v) {}

    void atacar() override {
        cout << nombre << " dispara una flecha a distancia." << endl;
    }
};
```

Captura 2: Clases derivadas

Texto:

Aquí se definen las clases Guerrero, Mago y Arquero, las cuales heredan de Personaje y sobrescriben el método atacar() para implementar comportamientos diferentes.

```
// MAIN
int main() {
    Guerrero g("Thor", 10, 100);
    Mago m("Merlin", 12, 80);
    Arquero a("Legolas", 11, 90);

    cout << "=== INFORMACIÓN ===" << endl;
    g.mostrarInfo();
    cout << endl;
    m.mostrarInfo();
    cout << endl;
    a.mostrarInfo();

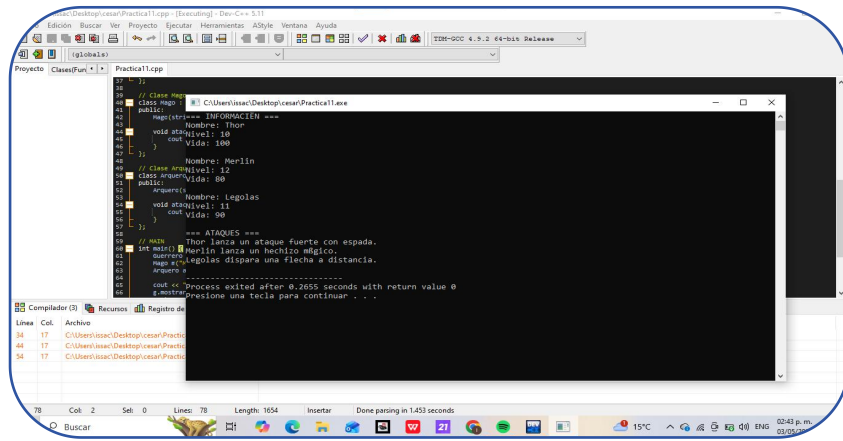
    cout << "\n=== ATAQUES ===" << endl;
    g.atacar();
    m.atacar();
    a.atacar();

    return 0;
}
```

Captura 3: Función main

Texto:

En esta sección se crean los objetos de cada tipo de personaje y se utilizan sus métodos para mostrar información y ejecutar ataques.

The screenshot shows the Dev-C++ IDE with a C++ program named 'Practical1.cpp' being executed. The program defines a base class 'Personaje' and three derived classes: 'Guerrero' (Warrior), 'Mago' (Mage), and 'Arquero' (Archer). Each class has attributes like 'Nombre' (Name), 'ataque' (attack), and 'vida' (health). The 'atacar()' method is overridden in each derived class to perform specific actions: 'Thor lanza un ataque fuerte con espada.' for the warrior, 'Merlin lanza un hechizo mágico.' for the mage, and 'Legolas dispara una flecha a distancia.' for the archer. The execution output in the console shows the initialization of these characters and the execution of their attack methods. The status bar at the bottom indicates 'Done parsing in 1.453 seconds'.

CAPTURAS DE EJECUCIÓN

Se muestra la ejecución del programa donde cada personaje presenta su información y realiza un tipo de ataque diferente.

EXPLICACIÓN

La herencia se utilizó mediante una clase base llamada Personaje, de la cual heredan las clases Guerrero, Mago y Arquero.

Esto permitió reutilizar los atributos y métodos comunes sin necesidad de repetir código. Cada clase derivada sobrescribe el método atacar() para definir su propio comportamiento.

La diferencia entre una clase base y una clase derivada es que la clase base contiene características generales, mientras que las derivadas especializan esas características.

CONCLUSIONES

En esta práctica se logró aplicar correctamente el concepto de herencia en C++, creando un sistema funcional de personajes.

Se comprendió cómo reutilizar código mediante una clase base y cómo modificar comportamientos en clases derivadas utilizando la sobrescritura de métodos.

Además, se observó la importancia de estructurar bien un programa para hacerlo más claro, organizado y fácil de mantener.